Binary File Handling in Java

29 June 2026 20:30

A binary file stores data in binary (zeros and ones) Rather than as human readable text. Unlike text files, binary files preserve the exact internal representation of data. Making them faster, more compact, and suitable for storing objects, images, audio, video and primitive data types.

For example:

Text File(student.txt)
101 Amit 92.5
102 Priya 88.0

The contents are readable

Binary File(student.dat)

010000001 0110101101 0111101110…

The content is not human readable. Sometime it displays symbols also, when we render it on different text editors.

Advantages of Binary Files

- Faster read or write operation.
- Occupy less storage.
- Preserve the exact data type.
- No conversion between text and numeric values.
- Suitable for storing Java objects.

Java Classes Used for Binary File Handling

Java provides several classes for binary file operations.

Class	Purpose
FileInputStream	**Reads binary data byte by byte.**
FileOutputStream	**Writes binary data byte by byte.**
BufferedInputStream	**Faster buffered reading.**
BufferedOutputStream	**Faster buffered writing.**
DataInputStream	**Reads primitive data types.**
DataOutputStream	**Writes primitive data types.**
ObjectInputStream	**Reads serialized objects.**
ObjectOutputStream	**Writes serialized objects.**
RandomAccessFile	**Reads or writes at any location.**

FileOutputStream: We used to write raw bytes into a binary file.

Class FileOutputStream

java.lang.Object
  java.io.OutputStream
    java.io.FileOutputStream

All Implemented Interfaces:

Closeable, Flushable, AutoCloseable

```
public class FileOutputStream
extends OutputStream
```

A file output stream is an output stream for writing data to a `File` or to a `FileDescriptor`. Whether or not a file is available or may be created depends upon the underlying platform. Some platforms, in particular, allow a file to be opened for writing by only one `FileOutputStream` (or other file-writing object) at a time. In such situations the constructors in this class will fail if the file involved is already open.

`FileOutputStream` is meant for writing streams of raw bytes such as image data. For writing streams of characters, consider using `FileWriter`.

API Note:

The `close()` method should be called to release resources used by this stream, either directly, or with the `try`-with-resources statement.

Implementation Requirements:

Subclasses are responsible for the cleanup of resources acquired by the subclass. Subclasses requiring that resource cleanup take place after a stream becomes unreachable should use `Cleaner` or some other mechanism.

Since:

1.0

Constructor Summary

Constructors

Constructor	Description
`FileOutputStream(File file)`	Creates a file output stream to write to the file represented by the specified `File` object.
`FileOutputStream(FileDescriptor fdObj)`	Creates a file output stream to write to the specified file descriptor, which represents an existing connection to an actual file in the file system.
`FileOutputStream(File file, boolean append)`	Creates a file output stream to write to the file represented by the specified `File` object.
`FileOutputStream(String name)`	Creates a file output stream to write to the file with the specified name.
`FileOutputStream(String name, boolean append)`	Creates a file output stream to write to the file with the specified name.

Method Summary

All Methods | **Instance Methods** | **Concrete Methods**

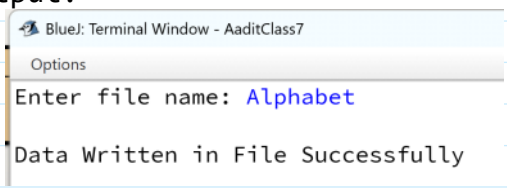
Modifier and Type	Method	Description
`void`	`close()`	Closes this file output stream and releases any system resources associated with this stream.
`FileChannel`	`getChannel()`	Returns the unique `FileChannel` object associated with this file output stream.
`final FileDescriptor`	`getFD()`	Returns the file descriptor associated with this stream.
`void`	`write(byte[] b)`	Writes `b.length` bytes from the specified byte array to this file output stream.
`void`	`write(byte[] b, int off, int len)`	Writes `len` bytes from the specified byte array starting at offset `off` to this file output stream.
`void`	`write(int b)`	Writes the specified byte to this file output stream.

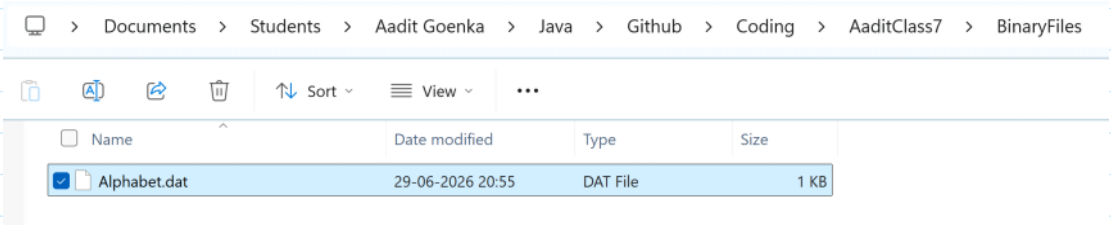
```
package FileHandling.BinaryFileHandling;
import java.util.*;
import java.io.*;
```

```
/**
 * Write a description of class WriteBinary here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class WriteBinary
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("\fEnter file name: ");
        String path = "./BinaryFiles/";
        String fileName = sc.nextLine();
        try(FileOutputStream fos = new
        FileOutputStream(path+fileName+".dat", true)){
            fos.write(65);
            fos.write(66);
            fos.write(67);
            fos.write(68);

            fos.close();
            System.out.print("\nData Written in File Successfully");
        }
        catch(FileNotFoundException fnfe){
            System.err.print("\nFile is not present on the specified
directory.");
        }
        catch(IOException ioe){
            System.err.print("\nInput/Output network jam.");
        }
    }
}
```

Output:





FileInputStream: Reads bytes from the binary file.

Class FileInputStream

java.lang.Object
  java.io.InputStream
    java.io.FileInputStream

All Implemented Interfaces:

Closeable, AutoCloseable

```
public class FileInputStream
extends InputStream
```

A `FileInputStream` obtains input bytes from a file in a file system. What files are available depends on the host environment.

`FileInputStream` is meant for reading streams of raw bytes such as image data. For reading streams of characters, consider using `FileReader`.

API Note:

The `close()` method should be called to release resources used by this stream, either directly, or with the `try`-with-resources statement.

Implementation Requirements:

Subclasses are responsible for the cleanup of resources acquired by the subclass. Subclasses requiring that resource cleanup take place after a stream becomes unreachable should use `Cleaner` or some other mechanism.

Since:

1.0

Constructor Summary

Constructors

Constructor	Description
`FileInputStream(File file)`	Creates a `FileInputStream` to read from an existing file represented by the `File` object `file`.
`FileInputStream(FileDescriptor fdObj)`	Creates a `FileInputStream` by using the file descriptor `fdObj`, which represents an existing connection to an actual file in the file system.
`FileInputStream(String name)`	Creates a `FileInputStream` to read from an existing file named by the path name `name`.

Method Summary

All Methods | **Instance Methods** | **Concrete Methods**

Modifier and Type	Method	Description
`int`	`available()`	Returns an estimate of the number of remaining bytes that can be read (or skipped over) from this input stream without blocking by the next invocation of a method for this input stream.
`void`	`close()`	Closes this file input stream and releases any system resources associated with the stream.
`FileChannel`	`getChannel()`	Returns the unique `FileChannel` object associated with this file input stream.
`final FileDescriptor`	`getFD()`	Returns the `FileDescriptor` object that represents the connection to the actual file in the file system being used by this `FileInputStream`.
`int`	`read()`	Reads a byte of data from this input stream.
`int`	`read(byte[] b)`	Reads up to `b.length` bytes of data from this input stream into an array of bytes.
`int`	`read(byte[] b, int off, int len)`	Reads up to `len` bytes of data from this input stream into an array of bytes.
`byte[]`	`readNBytes(int len)`	Reads up to a specified number of bytes from the input stream.
`long`	`skip(long n)`	Skips over and discards `n` bytes of data from the input stream.

read

```
public int read()
        throws IOException
```

Reads a byte of data from this input stream. This method blocks if no input is yet available.

Specified by:

read in class InputStream

Returns:

the next byte of data, or -1 if the end of the file is reached.

Throws:

IOException - if an I/O error occurs.

```
package FileHandling.BinaryFileHandling;
import java.util.*;
import java.io.*;


/**
 * Write a description of class ReadBinary here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ReadBinary
{
    public static void main(String arga[]){
        Scanner sc = new Scanner(System.in);
        System.out.print("\fEnter file name: ");
        String path = "./BinaryFiles/";
        String fileName = sc.nextLine();
        int ch;
        try(FileInputStream fis = new FileInputStream(path+fileName)){
            while((ch = fis.read()) != -1){
                System.out.print((char)ch);
            }
        }
        catch(IOException ioe){
            System.out.print("\nNetwork error!");
        }
    }
}
```

Output: